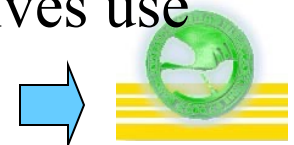


Chapter 19

Recovery System



- **Transaction failure :**
 - **Logical errors:** transaction cannot complete due to some internal error condition
 - **System errors:** the database system must terminate an active transaction due to an error condition (e.g., deadlock)
- **System crash:** a power failure or other hardware or software failure causes the system to crash.
- **Fail-stop assumption:** non-volatile storage contents are assumed to not be corrupted by system crash
- **Disk failure:** a head crash or similar disk failure destroys all or part of disk storage
 - Destruction is assumed to be detectable: disk drives use checksums to detect failures



- Recovery algorithms are techniques to ensure database consistency and transaction atomicity and durability despite failures
- Recovery algorithms have two parts
 - Actions taken during normal transaction processing to ensure enough information exists to recover from failures
 - Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability



- **Volatile storage** (易失) :does not survive system crashes, e.g. main memory, cache memory
- **Nonvolatile storage**:survives system crashes
- e.g. disk, tape, flash memory, non-volatile (battery backed up) RAM
- **Stable storage**:a mythical form of storage that survives all failures, approximated by maintaining multiple copies on distinct nonvolatile media, e.g. RAID



- **Physical blocks** are those blocks residing on the disk.
- **Buffer blocks** are the blocks residing temporarily in main memory.
- Block movements between disk and main memory are initiated through the following two operations:
 - **input**(B) transfers the physical block B to main memory.
 - **output**(B) transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- Each transaction T_i has its private work-area in which local copies of all data items accessed and updated by it are kept.
 - T_i 's local copy of a data item X is called x_i .



- We assume, for simplicity, that each data item fits in, and is stored inside, a single block.
- Transaction transfers data items between system buffer blocks and its private work-area using the following operations :
 - **read**(X) assigns the value of data item X to the local variable x_i .
 - **write**(X) assigns the value of local variable x_i to data item $\{X\}$ in the buffer block.
 - both these commands may necessitate the issue of an **input**(B_X) instruction before the assignment, if the block B_X in which X resides is not already in memory.



■ Transactions

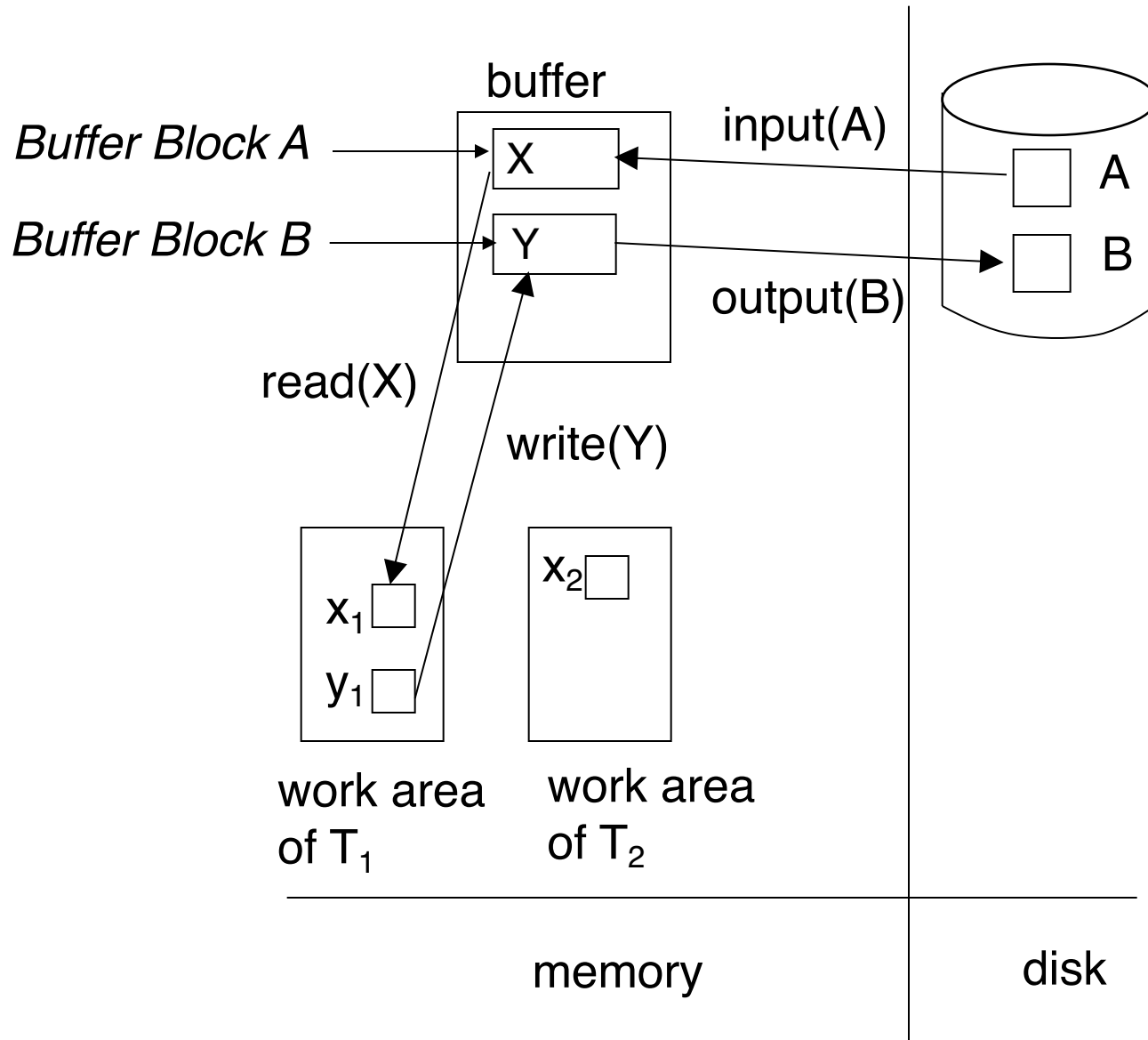
- Perform **read**(X) while accessing X for the first time;
- All subsequent accesses are to the local copy.
- After last access, transaction executes **write**(X).

■ **output**(B_X) need not immediately follow **write**(X).

System can perform the **output** operation when it deems fit.



Example of Data Access



- • Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state.
- • Consider transaction T_i that transfers \$50 from account A to account B ; goal is either to perform all database modifications made by T_i or none at all.
- • Several output operations may be required for T_i (to output A and B). A failure may occur after one of these modifications have been made but before all of them are made.

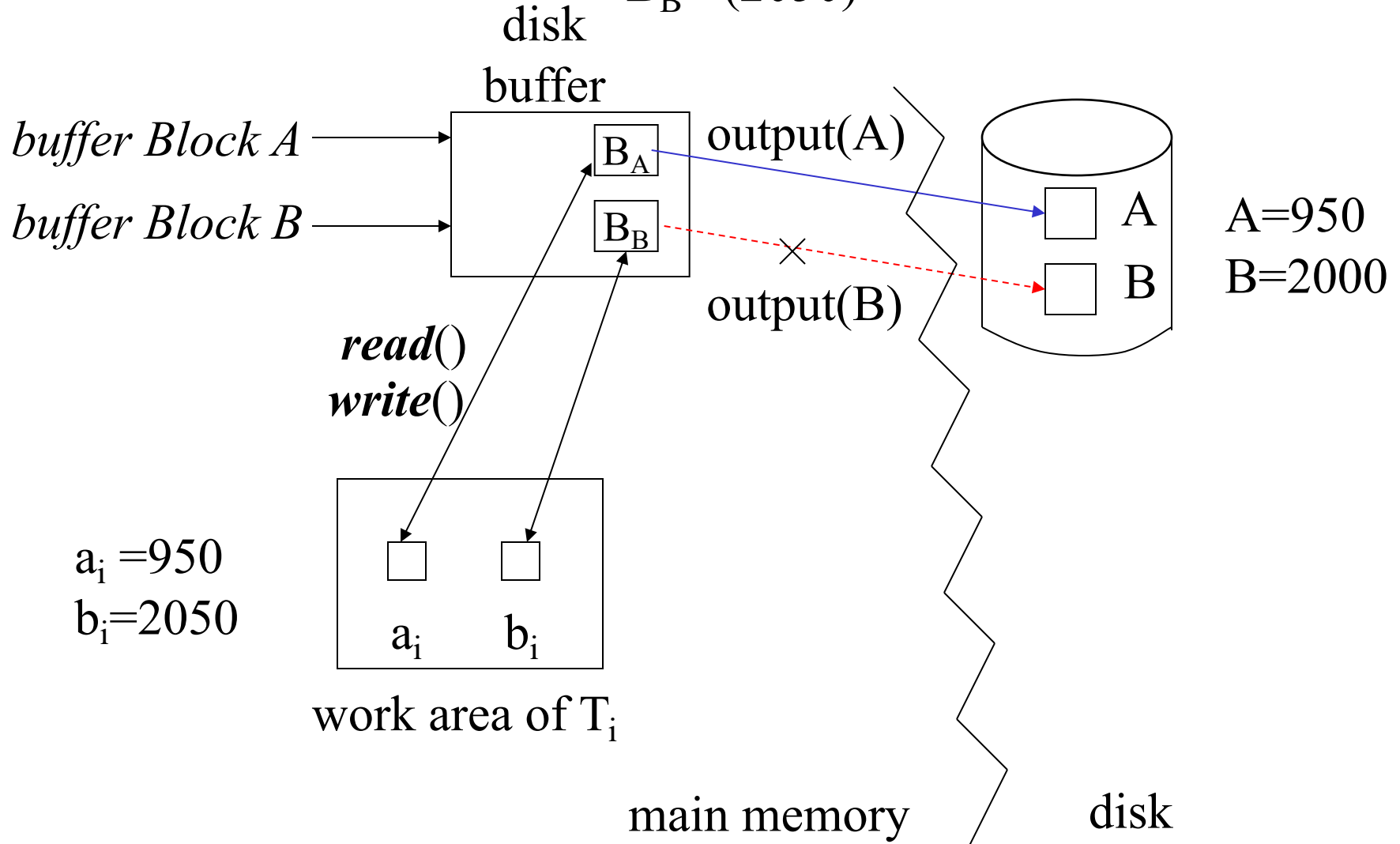


- Problem
 - when system failure occurs, the modifying of the database by transaction may leave the database in an inconsistent state.
- E.g. Consider transaction T_i that transfers \$50 from account A to account B , with initial values of A and B being \$1000, and \$2000
 - a **system crash** occurs during the execution of T_i , after $\text{output}(B_A)$ has taken place, but before $\text{output}(B_B)$ is executed
 - the values of A and B in working area, disk buffer and disk are shown on the next PPT



$$B_A = (950)$$

$$B_B = (2050)$$



Contents of memory and DB when system crash occurs

- after DBS recovers from system crash, if
 - T_i is not re-executed, DB is concurrently in an inconsistent in which $A=950$ and $B=2000$
 - T_i is re-executed, the DB will enter an inconsistent state in which $A=900$ and $B=2050$
- with respect to these two modifications of DB, i.e., $output(A)$ and $output(B)$, inconsistency occurs after $output(A)$ has been made but before all of these two modifications are made
- so, **recovery actions** are needed to be taken
- To ensure DB consistency, the goal is either to perform all database modifications made by T_i or none at all
 - atomicity of T_i should be guaranteed despite failures



■ To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself.

■ We study two approaches:

- **log-based recovery**, and

- **shadow-paging**

■ We assume (initially) that transactions run serially, that is, one after the other.



Log-Based Recovery

- The log is a sequence of **log records**, and maintains a record of update activities on the database.

A **log** is kept on stable storage.

- $\langle T_i \text{ start} \rangle$
 $\langle T_i, X, V_1, V_2 \rangle$
 $\langle T_i \text{ commit} \rangle$
 $\langle T_i \text{ abort} \rangle$

- Two approaches using logs
 - Deferred database modification
 - Immediate database modification



Database Modification

- The **database modification** scheme allows database updates of an uncommitted transaction to be made as the writes are issued, $\langle T_i, X, V_1, V_2 \rangle$
- Update log record must be written *before* database item is written
 - ★ We assume that the log record is output directly to stable storage
 - ★ Can be extended to postpone log record output, so long as prior to execution of an **output**(B) operation for a data block B , all log records corresponding to items B must be flushed to stable storage
- Output of updated blocks can take place at any time before or after transaction commit



Database Modification Example

Log

Write

Output

$\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 1000, 950 \rangle$

$\langle T_0, B, 2000, 2050 \rangle$

$A = 950$
 $B = 2050$

$\langle T_0 \text{ commit} \rangle$

$\langle T_1 \text{ start} \rangle$

$\langle T_1, C, 700, 600 \rangle$

$C = 600$

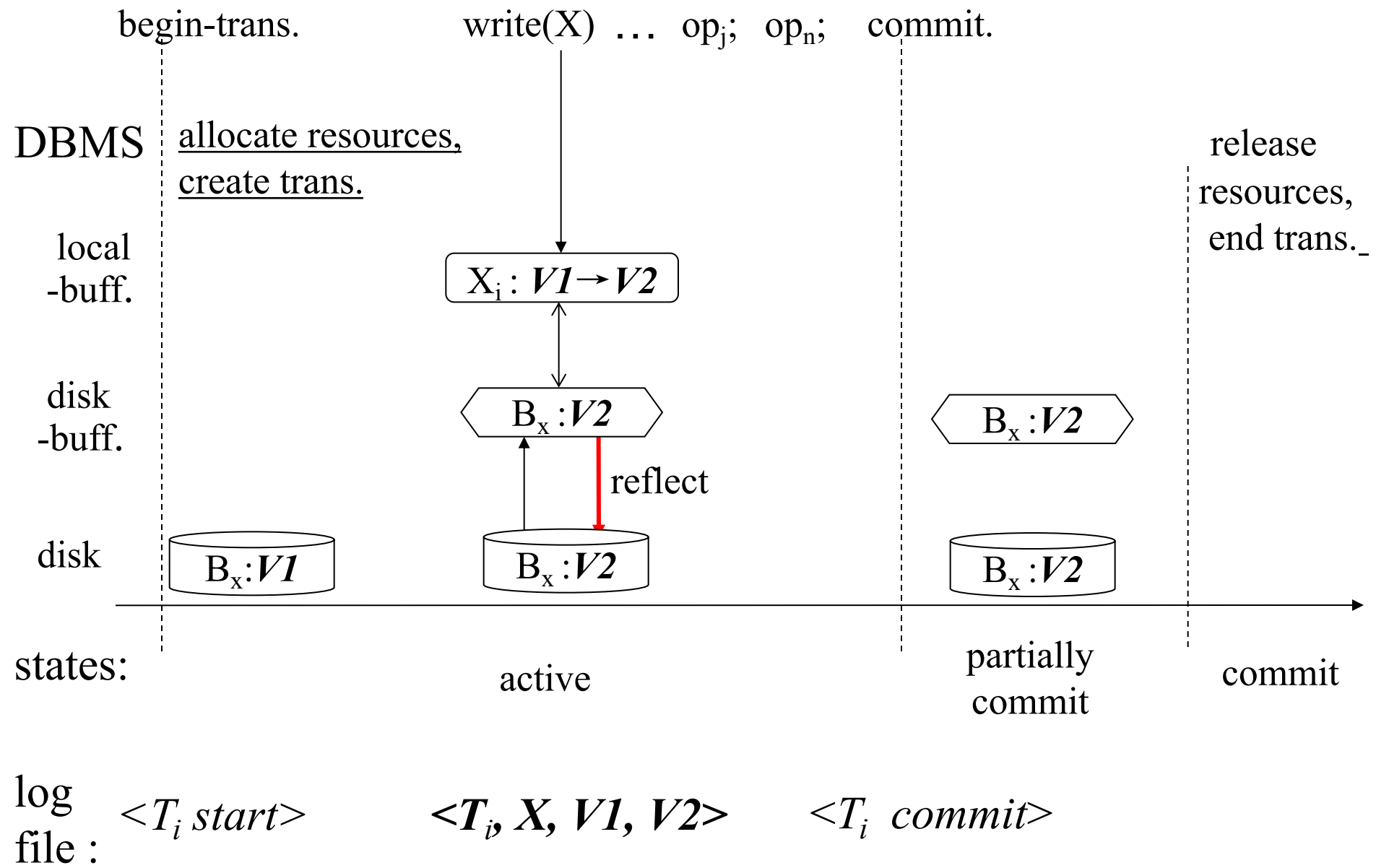
B_B, B_C

$\langle T_1 \text{ commit} \rangle$

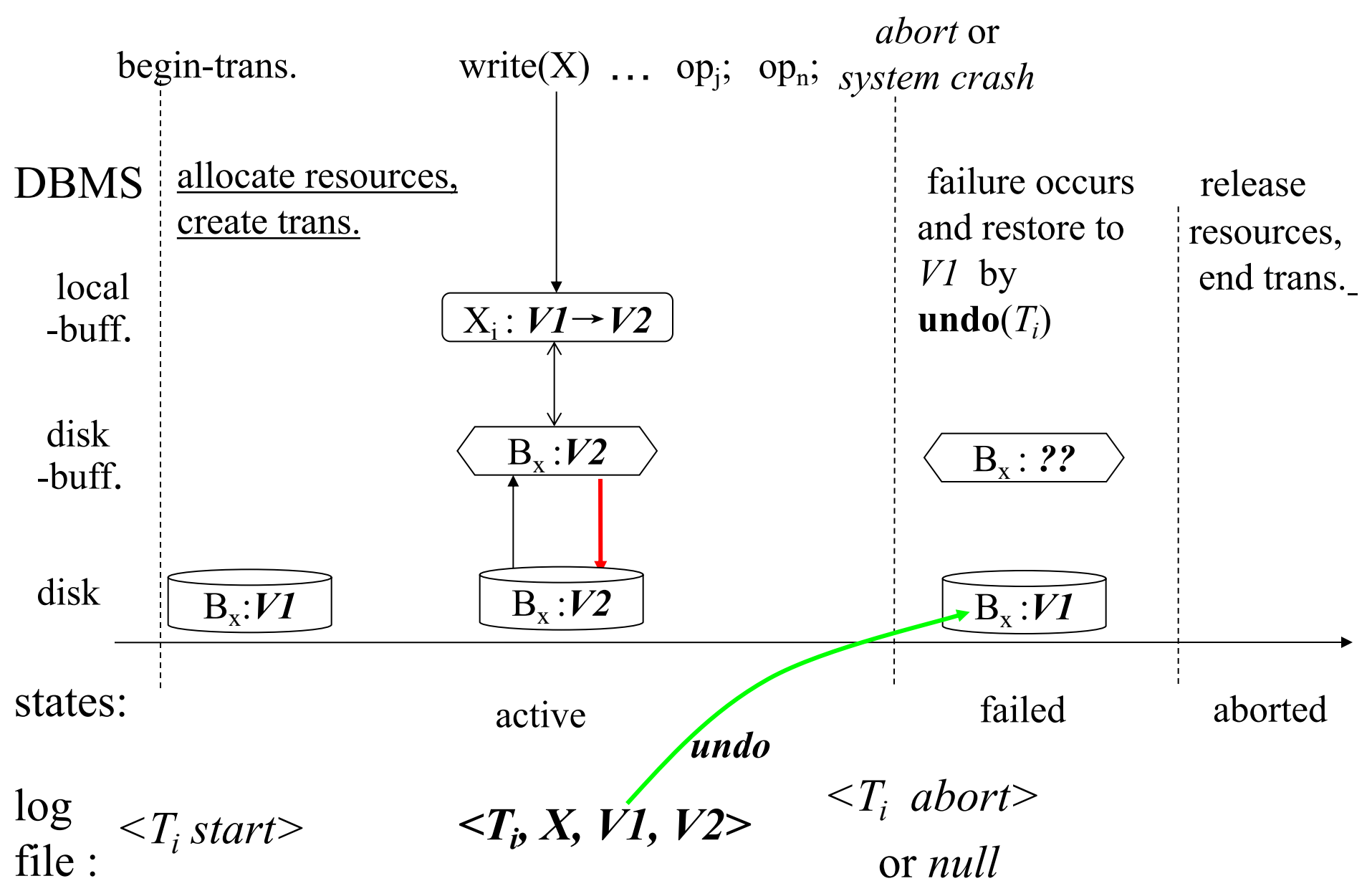
B_A

- Note: B_X denotes block containing X .

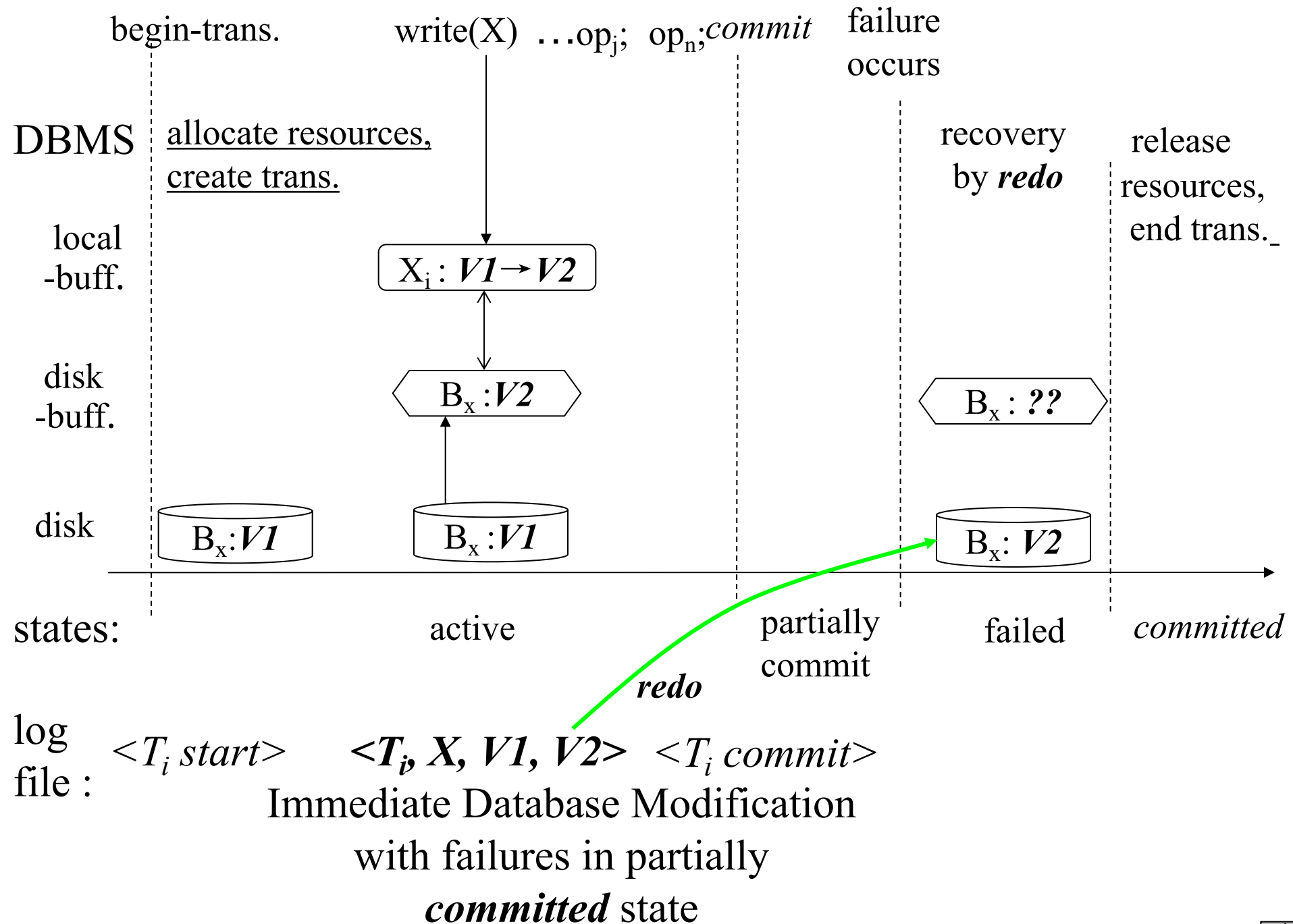




Database Modification without failure



database modification with failures in active
state



- Recovery procedure has two operations instead of one:
 - ★ **undo**(T_i) restores the value of all data items updated by T_i to their old values, going backwards from the last log record for T_i
 - ★ **redo**(T_i) sets the value of all data items updated by T_i to the new values, going forward from the first log record for T_i
- Both operations must be **idempotent**
 - ★ That is, even if the operation is executed multiple times the effect is the same as if it is executed once



- The **undo** and **redo** operations are used in several different circumstances:
 - The **undo** is used for transaction **rollback** during normal operation (in case a transaction cannot complete its execution due to some **logical error**).
 - The **undo** and **redo** operations are used during recovery from **failure**.
 - Undo operations are performed first, then redo operations
- We need to deal with the case where during *recovery from failure* **another failure occurs** prior to the system having fully recovered——连续发生多个failure



Transaction rollback (during normal operation)

- Let T_i be the transaction to be rolled back
- Scan log backwards from the end, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$
 - perform the undo by writing V_1 to X_j ,
 - write a log record $\langle T_i, X_j, V_1 \rangle$
 - such log records are called **compensation log records**
- Once the record $\langle T_i, \text{start} \rangle$ is found stop the scan and write the log record $\langle T_i, \text{abort} \rangle$



$T_0 : \dots \dots \text{write}(Q) \dots \text{write}(Q) \text{ op}_j; \dots \text{op}_n; \dots \dots \text{rollback ; recovery}$
*due to failure by **undo***

local
-buff.

$Q_0 : V1 \rightarrow V2$

$Q_1 : V2 \rightarrow V3$

disk
-buff.

$B_Q : V2$

$B_Q : V3$

$B_Q : V1$

$B_Q : V2$

$B_Q : V3$

undo(T_0)

$B_Q : V1$

$\langle T_0, Q, V1, V2 \rangle$

$\langle T_0, Q, V2, V3 \rangle$

log

$\langle T_0, Q, V2 \rangle$

$\langle T_0, Q, V1 \rangle$

$\langle T_0, \text{abort} \rangle$

Undo in case of a single transaction *rollback*

• When recovering after failure:

- Transaction T_i needs to be *undone* if the log
 - contains the record $\langle T_i \text{ start} \rangle$,
 - but does not contain either the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$.

— 发生failure时(因crash等外部原因), T_i 没有完成

- Transaction T_i needs to be redone if the log
 - contains the records $\langle T_i \text{ start} \rangle$
 - and contains the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$

— 发生failure时, T_i 逻辑上已经完成



Example

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

(a) undo (T_0): B is restored to 2000 and A to 1000.

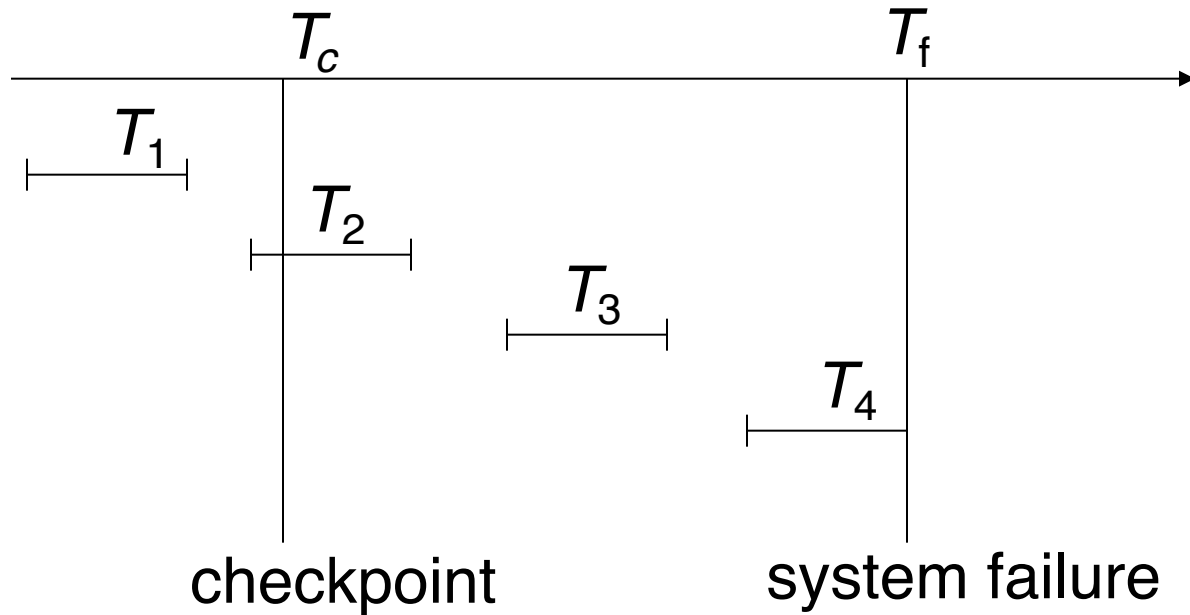
(b) undo (T_1) and redo (T_0): C is restored to 700, and then A and B are set to 950 and 2050 respectively.

(c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then C is set to 600



- Problems in recovery procedure as discussed earlier :
 1. searching the entire log is time-consuming
 2. we might unnecessarily redo transactions which have already output their updates to the database.
- Streamline recovery procedure by periodically performing **checkpointing**
 - Output all log records currently residing in main memory onto stable storage.
 - Output all modified buffer blocks to the disk.
 - Write a log record < **checkpoint** > onto stable storage.





- T_1 can be ignored (updates already output to disk due to checkpoint)
- T_2 and T_3 redone.
- T_4 undone



● *Recovery With Concurrent Transactions* *DataBase System Concepts*

- We modify the log-based recovery schemes to allow multiple transactions to execute concurrently.
 - ★ All transactions share a single disk buffer and a single log
 - ★ A buffer block can have data items updated by one or more transactions
- We assume concurrency control using strict two-phase locking;
 - ★ i.e. the updates of uncommitted transactions should not be visible to other transactions
- Logging is done as described earlier.
 - ★ Log records of different transactions may be interspersed in the log.



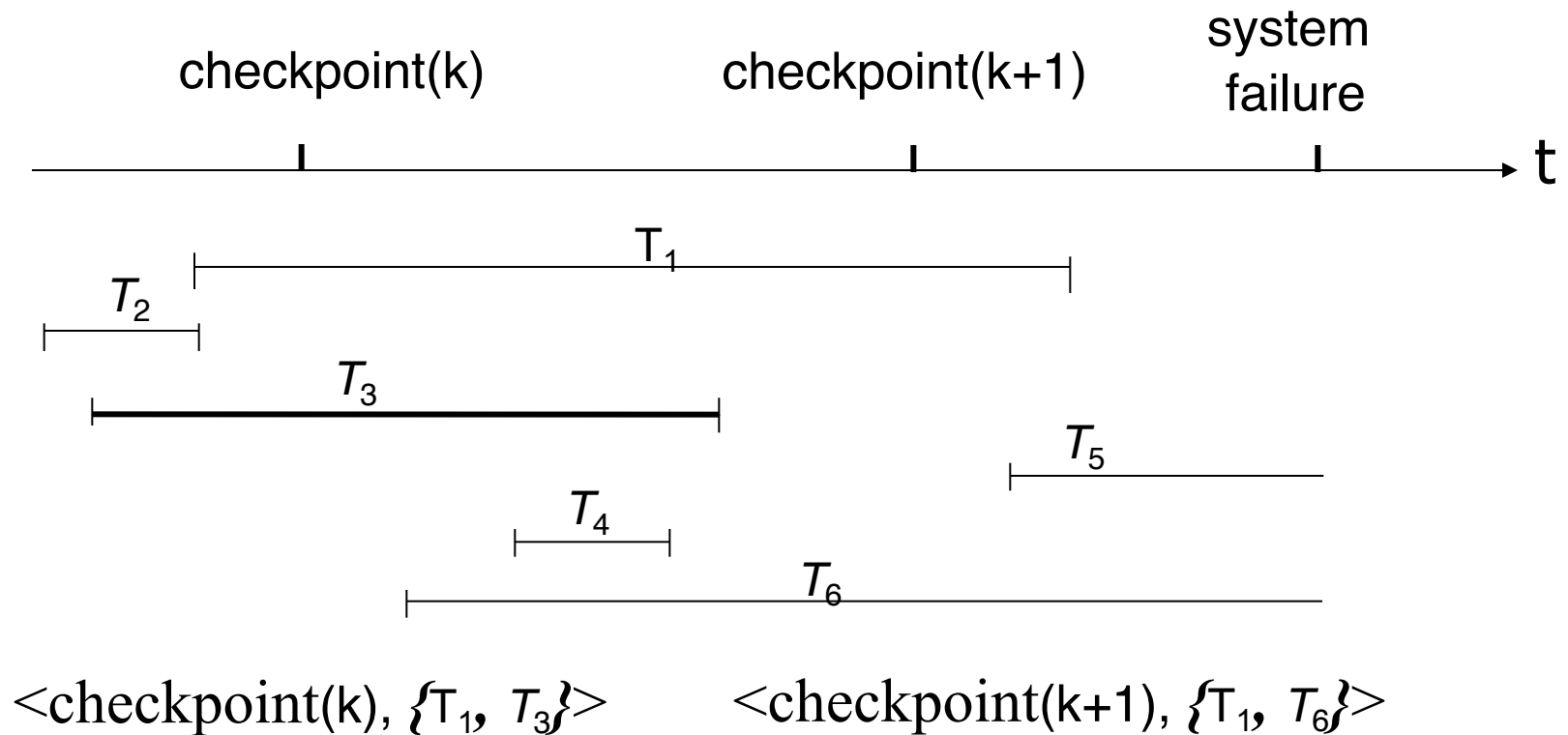
- Checkpoints are performed as before, except that the checkpoint log record is now of the form

< checkpoint L >

- where L is the list of transactions active at the time of the checkpoint

- When the system recovers from a crash, it first does the following:





log:...; $\langle \text{checkpoint}(k+1), \{T_1, T_6\} \rangle$; ...; $\langle T_5 \text{ start} \rangle$; ...; $\langle T_1 \text{ commit} \rangle$; ...

An example of checkpoint log

T₁:	T₂:	Recovery	log
<i>begin-transaction</i>			<T ₁ start>
read (A)			<T ₂ start>
A:= A-50	<i>begin-transaction</i>		<T ₁ , A, 1000, 950>
<i>write (A)</i>			
	read(A)		<checkpoint {T ₁ . T ₂ }>
	temp:= A*0.1		
		<i>checkpoint</i>	
	A := A- temp		<T ₂ , A, 950, 855>
	<i>write (A)</i>		
read (B)			<T ₁ , B, 2000, 2050>
B:= B+50			<T ₁ , commit>
<i>write(B)</i>			
<i>commit</i>			<checkpoint {T ₂ }>
	read (B)		
		<i>checkpoint</i>	
	B:= B + temp		<T ₂ , B, 2050, 2145>
	<i>write(B)</i>		<T ₂ ,B,2050>
			<T ₂ ,A,950>
	<i>rollback</i>		<T ₂ ,abort>

- if a transaction T has update a data item Q, no other transaction may update the same data item Q until T has committed or rolled back
- This requirement can be ensured by *strict* two-phase locking protocol



Transaction Rollback (during normal operation)

- No system crash, and no $\langle T_i, \text{committed} \rangle$ in the log file, so the operations of T_i should be **undo**
- Let T_i is the transaction to be rolled back, rolling back as follows
 - **scan log backwards from the end**, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$
 - perform the undo by writing V_1 to X_j ,
 - write a log record $\langle T_i, X_j, V_1 \rangle$
 - such log records are called **compensation log records**
 - Once the record $\langle T_i, \text{start} \rangle$ is found, stop the scan and write the log record $\langle T_i, \text{abort} \rangle$



Scanning the log backward is important, because T_i may update the data item Q several times; the “oldest” *update* $\langle T_i, Q, V_i, V_j \rangle$ by T_i should be employed to restore the data item Q



- Recovery from crash in two phases
 - Redo phase:
replay updates of all transactions by **scanning the log forward from the last checkpoint**, *whether they committed, aborted, or are incomplete*
 - Undo phase:
undo/rollback all incomplete transactions in the undo-list



- Step1. Find last **<checkpoint L>** record, and set the undo-list to L
 - the undo list records the transactions to be rolled back
- Step2. Scan forward from above **<checkpoint L>** record
 - whenever a record $\langle T_i, X_j, V_1, V_2 \rangle$ is found, or a read only record $\langle T_i, X_j, V_2 \rangle$ is found, redo it by writing V_2 to X_j ——最近检查点时正在进行、还未结束的事务
 - whenever a log record $\langle T_i \text{ start} \rangle$ is found, add T_i to undo-list——最近检查点之后开始的事务
 - whenever a log record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$ is found, remove T_i from undo-list



- Scan log **backwards from end**, to roll back all transactions in undo-list
 - whenever a log record $\langle T_i, X_j, V_1, V_2 \rangle$ is found where T_i is in undo-list, perform same actions as for transaction rollback
 1. perform undo by writing V_1 to X_j .
 2. write a log record $\langle T_i, X_j, V_1 \rangle$
 - whenever a log record $\langle T_i \text{ start} \rangle$ is found where T_i is in undo-list,
 1. write a log record $\langle T_i \text{ abort} \rangle$
 2. remove T_i from undo-list

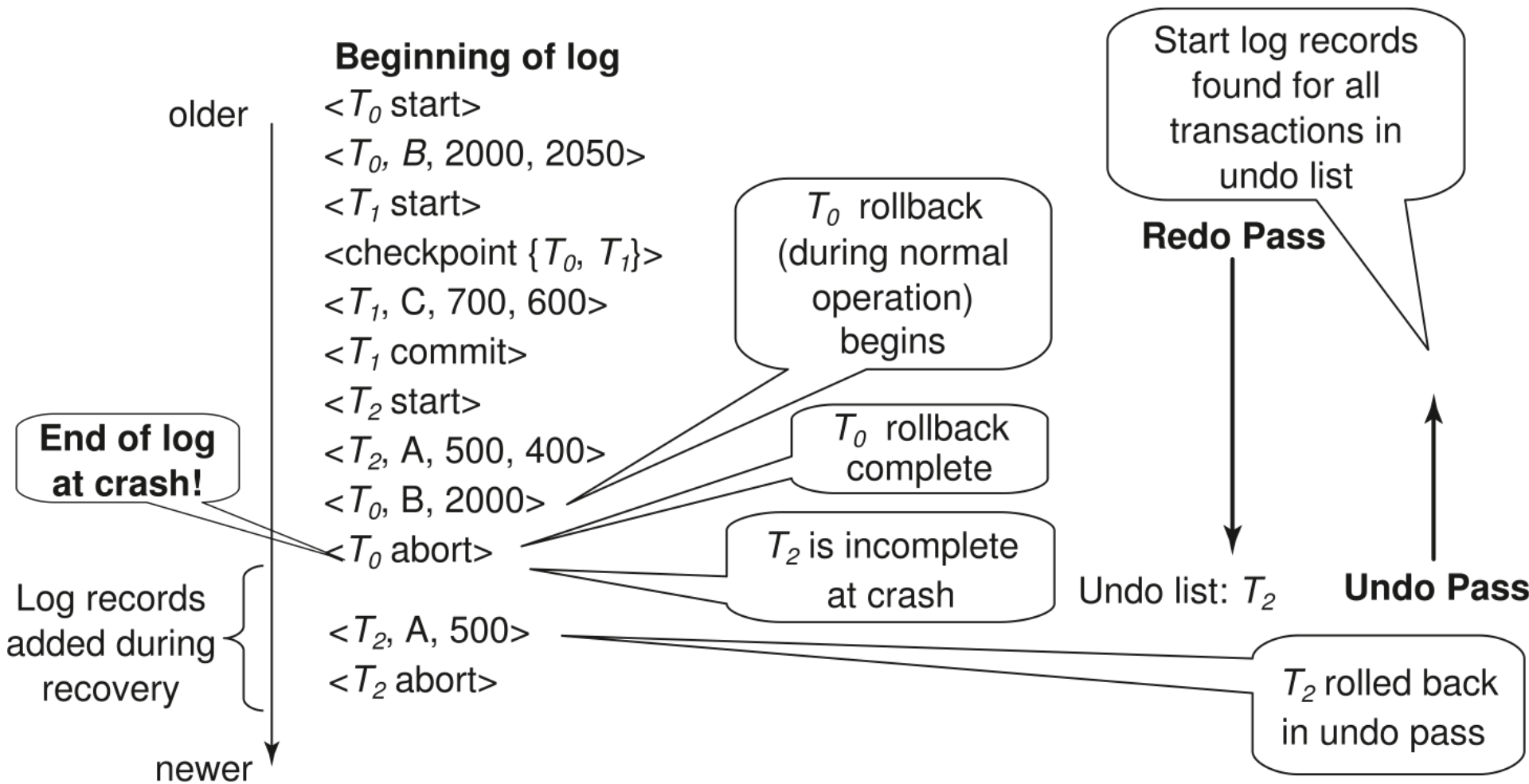


Undo Phase

- stop when undo-list is empty
i.e., $\langle T_i \text{ start} \rangle$ has been found for every transaction in undo-list

● After undo phase completes, normal transaction processing can commence





版本5中的老方法：
1.构造2 张表：
redo_list, undo_list
2.先undo，
3. 后redo

1. Initialize *undo-list* and *redo-list* to empty
2. Scan the log backwards from the end, stopping when the first **<checkpoint L>** record is found.
For each record found during the backward scan:
 - if the record is **< T_i commit>**, add T_i to *redo-list*
 - if the record is **< T_i start>**, then if T_i is not in *redo-list*, add T_i to *undo-list*
3. For every T_i in L , if T_i is not in *redo-list*, add T_i to *undo-list*

- Recovery now continues as follows:

1. Scan log backwards from most recent record, stopping when $\langle T_i \text{ start} \rangle$ records have been encountered for every T_i in undo-list.

- During the scan, perform **undo** for each log record that belongs to a transaction in undo-list.

2. Locate the most recent **<checkpoint L>** record.

3. Scan log forwards from the **<checkpoint L>** record till the end of the log.

- During the scan, perform **redo** for each log record that belongs to a transaction on redo-list



- Go over the steps of the recovery algorithm on the following log:

$\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 0, 10 \rangle$

$\langle T_0 \text{ commit} \rangle$

$\langle T_1 \text{ start} \rangle$ */* Scan at step 1 comes up to here */*

$\langle T_1, B, 0, 10 \rangle$

$\langle T_2 \text{ start} \rangle$

$\langle T_2, C, 0, 10 \rangle$

$\langle T_2, C, 10, 20 \rangle$

$\langle \text{checkpoint } \{T_1, T_2\} \rangle$

$\langle T_3 \text{ start} \rangle$

$\langle T_3, A, 10, 20 \rangle$

$\langle T_3, D, 0, 10 \rangle$

$\langle T_3 \text{ commit} \rangle$

****crash****

the following log:

<T₁ start>
<T₁, A, 100, 200>
<T₂ start>
<T₃ start>
<T₂ , B, 200, 100>
<checkpoint {T₁,T₂, T₃}>
<T₁ commit>
<T₃, C, 0, 100>
<T₃, C, 100, 400>
<checkpoint {T₂, T₃}>
<T₄ start>

<T₄, A, 200, 400>
<T₅ start>
<T₂, B, 100, 200>
<T₅, D, 300, 100>
<T₅, commit>
<T₄, A, 400, 250>
<T₃, C,0>
<T₄, commit>
<T₃,abort>
****Crash****

- **Log record buffering:** log records are buffered in main memory, instead of being output directly to stable storage.
 - Log records are output to stable storage when a block of log records in the buffer is full, or a **log force** operation is executed.
- Log force is performed to commit a transaction by forcing all its log records (including the commit record) to stable storage.
- Several log records can thus be output using a single output operation, reducing the I/O cost.



- The rules below must be followed if log records are buffered:
 - Log records are output to stable storage in the order in which they are created.
 - Transaction T_i enters the commit state only when the log record $\langle T_i \text{ commit} \rangle$ has been output to stable storage.
 - Before a block of data in main memory is output to the database, all log records pertaining to data in that block must have been output to stable storage.
 - This rule is called the **write-ahead logging** or **WAL** rule
 - Strictly speaking WAL only requires undo information to be output



- Database maintains an in-memory buffer of data blocks
 - When a new block is needed, if buffer is full an existing block needs to be removed from buffer
 - If the block chosen for removal has been updated, it must be output to disk
- As a result of the write-ahead logging rule, if a block with uncommitted updates is output to disk, log records with undo information for the updates are output to the log on stable storage first.



- No updates should be in progress on a block when it is output to disk. Can be ensured as follows.
 - Before writing a data item, transaction acquires exclusive lock on block containing the data item
 - Lock can be released once the write is completed.
 - Such locks held for short duration are called **latches**.
 - Before a block is output to disk, the system acquires an exclusive latch on the block
 - Ensures no update can be in progress on the block



<T₁ start>

<T₁, A, 0, 20>

<T₁ commit>

<T₂ start>

<T₃ start>

<checkpoint {T₂, T₃}>

<T₂ B, 0, 10>

<T₃, C, 0, 20>

<T₃, C, 20, 40>

<T₄ start>

<T₂,B,10,40>

<checkpoint {T₂, T₃,T₄}>

<T₅ start>

<T₅,D,20,30>

<T₄, A, 20, 50>

<T₄,commit>

<T₃, C, 20>

< T₅,commit>

<T₃, C, 0>

<T₃,abort>

****Failure****